



Programación de guiones

Manolo Pérez Aixendri

Desarrollo de Videojuegos- Grado de Multimedia

ETSE –Universitat de Valencia

PROGRAMACIÓN de GUIONES

- Introducción
- Clases
- MonoBehaviour
- Ciclo de ejecución
- Gestión de Escenas
- Persistencia de Datos
- Inputs
- Objetos persistentes entre escenas
- Corrutinas
- Ejemplos:
 - Semáforo
 - Torre de Hanoi

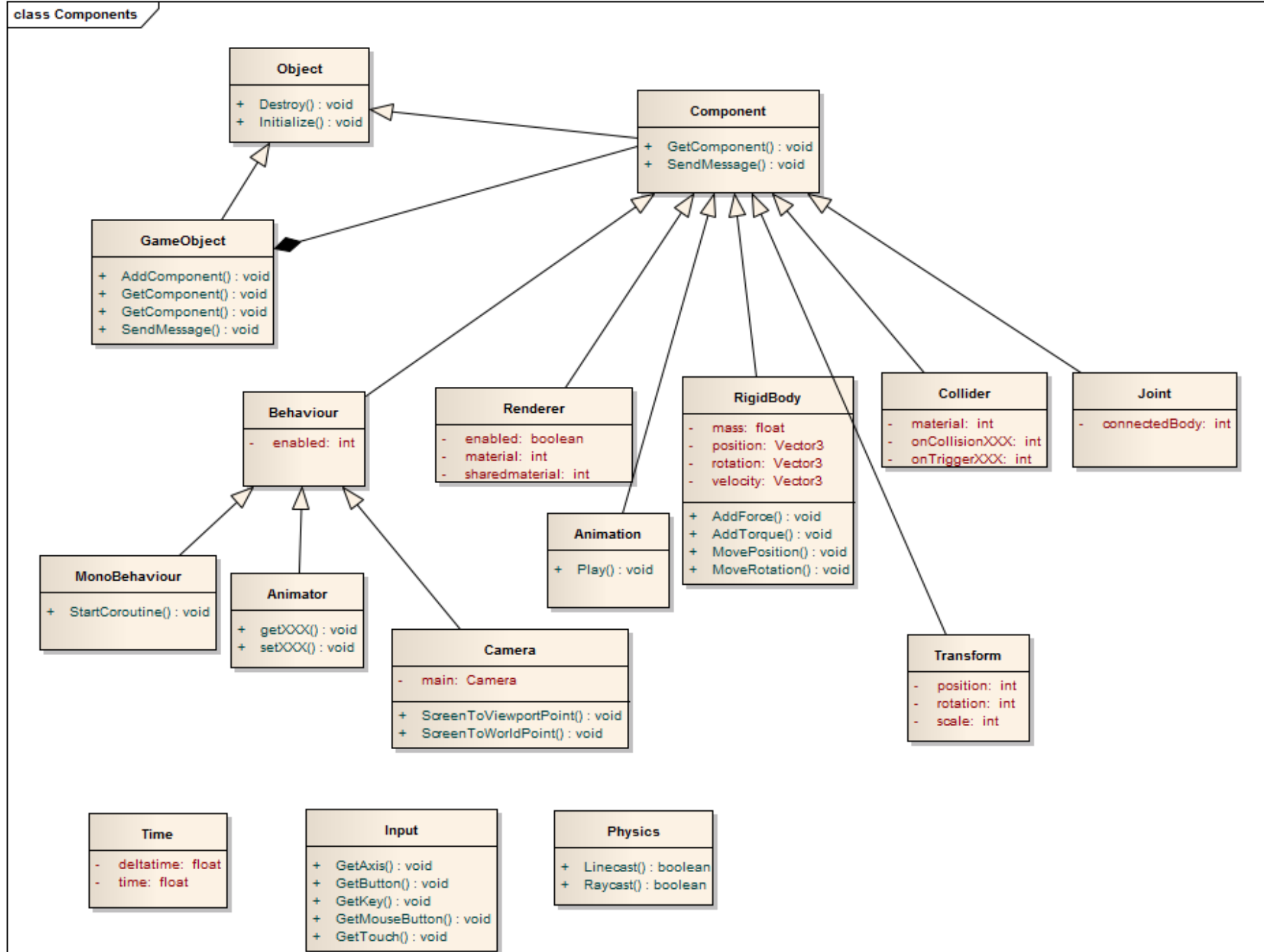
Introducción

- Permiten el control del comportamiento de los GameObject
- Programación en C#
- <http://docs.unity3d.com/es/current/Manual/ScriptingSection.html>
- <http://docs.unity3d.com/es/current/ScriptReference/index.html>

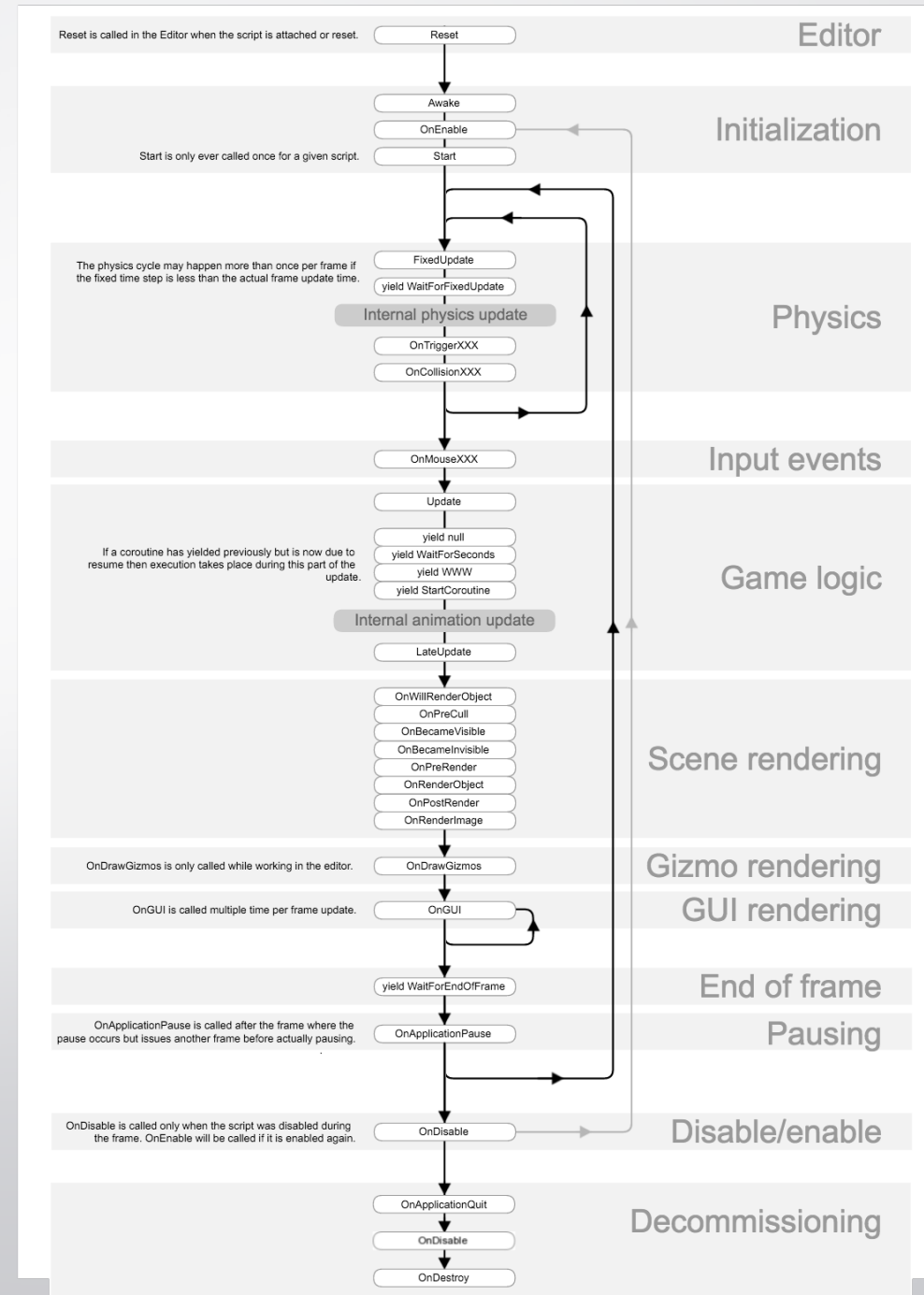
MonoBehaviour

- La clase base de todos los scripts de Unity
- Principales métodos: Start() y Update()
- Principales atributos: enabled, gameObject, tag, name y transform
- Acceder a otro Script → GetComponent<tipoScript>()
- Variables públicas visibles en Inspector
- Gestión de Mensajes:
 - <http://docs.unity3d.com/ScriptReference/MonoBehaviour.html>

Classes



Ciclo de ejecución



Gestión de Escenas

- `UnityEngine.SceneManagement.Scene`
 - **buildIndex**: Índice en BuildSettings. -1 si se carga de un AssetBundle
 - **isDirty**: Si la escena ha sido modificada
 - **isLoading**: Si la escena ha sido cargada
 - **Name**: Nombre de la escena
 - **Path**: Path relativo de la escena desde el directorio proyecto.
 - **rootCount**: Número de gameObject en la raíz de la escena
 - **GetRootGameObjects()**: Vector con los gameObjects en la raíz de la escena

Gestión de Escenas

UnityEngine.SceneManagement.SceneManager

- **sceneCount**: Número de escenas cargadas.
- **sceneCountBuildSettings**: Número de escenas incluidas en BuildSettings.
- **CreateScene()**: Crea una escena vacía.
- **GetActiveScene**: Devuelve la escena activa.
- **GetSceneAt-ByName-ByPath**: Obtiene una escena por índice/nombre/path.

Gestión de Escenas

UnityEngine.SceneManagement.SceneManager

- **LoadScene** y **LoadSceneAsync**: Carga una escena. Si es Async lo hace de manera asíncrona.
- **MergeScenes**: Junta una escena escena1 en una escena2. Al unir las, escena1 es destruida.
- **MoveGameObjectToScene**: Mueve un gameObject raíz de una escena a otra
- **SetActiveScene**: Activa la escena que recibe como parámetro. Debe estar previamente cargada
- **UnloadScene**: Descarga una escena.

Persistencia de Datos

- Queremos datos que persistan entre distintas ejecuciones. Existen varias soluciones:
 - Base de Datos: No integrado en Unity.
 - Ficheros → Carpetas específicas → `Application.persistentDataPath`
 - Registro de Sistema → `PlayerPrefs`

Persistencia de Datos

- UnityEngine.PlayerPrefs
 - Conjunto de Clave-Valor

<u>DeleteAll</u>	Borra todas las claves.
<u>DeleteKey</u>	Borra la clave que recibe como parámetro.
<u>GetFloat</u>	Devuelve el valor float de una clave.
<u>GetInt</u>	Devuelve el valor int de una clave.
<u>GetString</u>	Devuelve el valor strig de una clave.
<u>HasKey</u>	Devuelve true si existe una clave.
<u>Save</u>	Salva a disco el conjunto de claves-valores.
<u>SetFloat</u>	Guarda una clave con valor tipo float.
<u>SetInt</u>	Guarda una clave con valor tipo int.
<u>SetString</u>	Guarda una clave con valor tipo string.

↳ Son

Inputs

- Gestión de entradas de distintos sensores:
 - Teclado, ratón, joystick
 - Acelerómetro, giroscopio, brújula
 - Configurable desde Edit → Project Settings → Input

Inputs

- `GetAxis()`
- `GetButton()/GetButtonUp/Down()`
- `GetKey()/GetKeyUp/Down()`
- `GetMouseButton()/GetMouseButtonUp/Down()`
- `touchCount/getTouch()`

Objetos persistentes entre escenas

- A veces es necesario guardar información entre escenas en un GameObject
- pero los GameObjects de una escena se destruyen al cargar una nueva.
- DontDestroyOnLoad()
- Si recargamos la escena con un GameObject no destruible → duplicado

Objetos persistentes entre escenas

- Posible solución:

```
public class NoDestructible: MonoBehaviour
{
    static NoDestructible nd=null;

    void Awake()
    {
        if (!nd)
        {
            nd = this;
            DontDestroyOnLoad(gameObject);
        }
        else
            Destroy(gameObject);
    }
}
```

Corrutinas

- Supongamos el siguiente código:

```
void Fade()
```

```
{
```

```
    for (float f = 1f; f >= 0; f -= 0.1f)
```

```
    {
```

```
        Color c = renderer.material.color; c.a = f;
```

```
        renderer.material.color = c;
```

```
    }
```

```
}
```

```
void Update()
```

```
{
```

```
    if (Input.GetKeyDown("f"))
```

```
        Fade();
```

```
}
```


Corrutinas

- Solución:

`IEnumerator Fade()`

```
{  
    for (float f = 1f; f >= 0; f -= 0.1f)  
    {  
        Color c = renderer.material.color; c.a = f;  
        renderer.material.color = c;  
        yield return null;  
    }  
}
```

}

`void Update()`

{

`if (Input.GetKeyDown("f"))`

`StarCoroutine(Fade());`

}

→ se guarda

Corrutinas

- Solución:

```
Ienumerator Fade()
```

```
{
```

```
    for (float f = 1f; f >= 0; f -= 0.1f)
```

```
    {
```

```
        Color c = renderer.material.color; c.a = f;    }
```

```
        renderer.material.color = c;
```

```
        yield return WaitForSeconds(.1f);;
```

```
    }
```

```
}
```

```
void Update()
```

```
{
```

```
    if (Input.GetKeyDown("f"))
```

```
        StarCoroutine(Fade());
```

```
}
```

→ Se espera 0.1 seg.

Para evitar hacerlo rápido (120hz vs 60)

Corrutinas

```
void Start()
{
    print("Starting " + Time.time); 1
    StartCoroutine(WaitAndPrint(2.0F));
    print("Before WaitAndPrint Finishes " + Time.time); 2
}

IEnumerator WaitAndPrint(float waitTime)
{
    yield return new WaitForSeconds(waitTime);
    print("WaitAndPrint " + Time.time); 3
}
```

have give se
espera

```
IEnumerator Start()
{
    print("Starting " + Time.time); 1
    yield return StartCoroutine(WaitAndPrint(2.0F)); 2
    print("Done " + Time.time); 3
}

IEnumerator WaitAndPrint(float waitTime)
{
    yield return new WaitForSeconds(waitTime);
    print("WaitAndPrint " + Time.time);
}
```

Ejemplos

- Semáforo
- Torrete